

Writing sample: Legacy Code

Project brief

Project topic	Legacy Code
Client	DeepSource
Writer	Sara L. Santillan
Deliverable	1 article
Length	1800+ words
Time undertaken	10-15 hours to conceptualize, write, test (on DeepSource repositories), edit, and deploy
Project date	August 2021

The full article appears below:

What is Legacy Code?

Code has been around for a long time – more than half a century, in fact. In that span of time, operating systems have come and gone, languages have evolved, and software has been released, maintained, abandoned and sometimes even rediscovered. A programmer is likely to run into legacy code at some point in their career; bugfixing, porting, extending, or refactoring legacy code is *not* a niche task within the industry. Thus, learning *what* legacy code is and *how* to approach it is useful knowledge for any programmer who wants to level up their coding skills.

Common definitions of legacy code and their limitations

Ask ten developers what legacy code is, and you're likely to get ten different answers. Definitions abound online; Stack Overflow has [two threads](#) (one is a [duplicate](#) which still attracted answers) with a total of over twenty answers.

Legacy code is frequently and simply defined as "old code." But this amorphous description doesn't add any insight into the quality or functionality of the code, nor does it tell us what we're supposed to do with it. If code is "broken" at least we know what to do: debug it, fix it, and test it. But what do we do with old code? Do we want to make "old" code into "new" code? And when is code considered old? Is code old after five years or five days? Thus, this definition varies from person to person, and isn't very useful.

Legacy code is also defined as code in environments that are no longer supported or maintained. This legacy code, however, is still *useful* to some business or entity. This definition is broad, but it does imply what we're supposed to *do* with legacy code: update it, refactor it, port it, and/or migrate it. It also establishes a more definite metric: software and/or hardware vendors will usually publish the end-of-life dates for operating systems and other technologies.

Both definitions above, commonly seen in books and online, provide two perspectives on legacy code: a **micro perspective**, the way a programmer would personally assess code they're encountering ("this is old code" or "this is hard to change") and a **macro or high-level perspective**, the way a company or project manager would assess codebases fit for migration, expansion, porting, or some other update ("this code is unsupported by some vendor or operating system"). With these two perspectives in mind, we can gain a fuller picture of legacy code.

Legacy code from a project management perspective

From the use of the word "[legacy](#)" itself, legacy code is code that is associated with, or was carried over from, a previous time. A concrete example is the use of [COBOL in unemployment systems in the US](#), which caused a delay in issuing stimulus checks during the pandemic. As mentioned in the article, Colorado's Department of Labor and Employment itself had a project to migrate away from COBOL, as the old system struggled under the load of people trying to file for their benefits. COBOL, a programming language used widely in the 1970s, is no longer taught in most schools or used in business applications. In the case above, the legacy code in the unemployment processing system was due for a migration.

Legacy code still serves a purpose as seen in the example above. Users depend on these codebases. Otherwise, entities – corporations, government bodies, etc. – wouldn't make an effort to *identify* legacy code and change it. Thus the code is *inherited* by a team, who will update, port, or migrate the functionality to a new system, and technical specifications are a part of this process.

Legacy code from a programmer's perspective

A quick google search defines legacy code as "code you didn't write yourself," and "code that's difficult to understand and/or change," all coming from personal experiences. These definitions, while vague and raw, do capture the reality of working with legacy code. But *why* is this the case? Why is it "hard"?

An enduring definition of legacy code comes from Michael Feathers in his book [Working Effectively with Legacy Code](#) where he states that *legacy code is code without tests*. This includes code you've written as recently as last week, if it doesn't have tests of any sort. The reasoning in the book is as follows:

Legacy code is frequently (though not always) still in use: this makes it difficult to change, because changes can introduce unwanted side effects while the show must go on. The goal is to keep the legacy code's behavior without bugs or side effects. A check of some sort is needed – and tests are that check!

Without tests, it becomes difficult to understand the intent of the code, or what it's supposed to do. Thus, we arrive at those google results from earlier stating that, legacy code is "code that's difficult to understand and/or change."

In sum

- From a high-level, macro perspective, legacy code is valuable and often still in use for some business case. When the system(s) or environment(s) the code runs in are no longer maintained or supported, the code is identified as legacy and updated. The code is often *inherited* by a different team; sometimes it's no longer possible to contact the original programmers.
- From a more specific, programmer perspective, legacy code is difficult to change, understand, and work with. According to Feathers, this is because legacy code is code without tests that would assist with comprehension, refactoring, and the like.

Problems with Legacy Code

Here are some issues you may run into when handling a legacy codebase:

- **It may be difficult to comprehend.** It is harder to understand code you didn't write. Compounding this issue are several factors:
 - The original team or author of the code may be unavailable.
 - Documentation of the code or technical specification may also be unavailable.
 - The language is unfamiliar to the porting/updating/migrating team. This is especially [true](#)

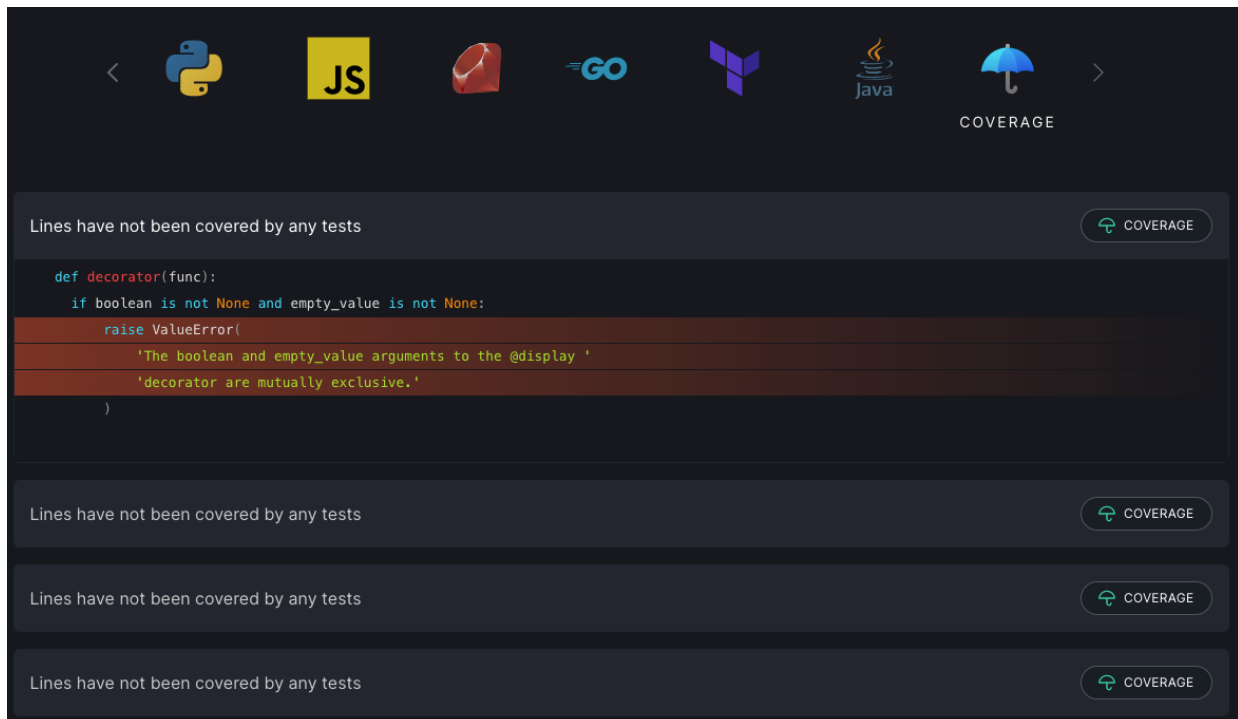
for code in large banks, airlines, and government offices, which were computerized in the 1970s-1980s.

- The original system environment (i.e. compilers, operating system) can no longer be recreated or emulated.
- **It may be difficult to test or inspect.** Related to the above, a programmer may have a hard time experimenting with the legacy system if they're unable to set up a development environment. End to end tests on legacy systems may also be difficult because the only existing codebase and data are always "in production" – programmers wouldn't be able to use fixtures, (also known as dummy/test data), to validate their understanding of how the codebase works.
- **It may be riddled with patchwork fixes.** All code over time accumulates *technical debt*, which sometimes ends up in production. This technical debt remains in the system, and may obscure the original intent of some piece of code.

Good Programming Practices for Legacy Code

- **Read the documentation, if available.** Write the documentation while reviewing the code, if none was provided. This increases comprehension without touching the codebase. Look out for the following:
 - *What coding conventions were outlined in the documentation?* This allows the team to decide whether or not they will update (change) the coding conventions or keep them. They can also calibrate their linter, formatter, and/or analyzer tools to check both the legacy code by its own rules, as well as the updated code.
 - *What frameworks and/or architecture were used in the code?* This will shed light on the intent of the overall code, which will trickle down into specific functions and modules.
 - DeepSource's Transformers enable teams to adhere to style conventions at the repository level. The code formatting, pull requesting and committing are all automated cleanly, in such a way that the commit history is preserved. Thus, changes made by the tool are both automatic and transparent, easily visible in the commit history.
- **Write tests.**
 - Michael Feathers suggests two kinds to write: *unit tests*, preferably automated, which can be run before any commit to make sure that changes haven't broken anything, and *characterization tests*, which are tests that document the code's *actual* behavior, not "the behavior you *wish* your system had" (as stated [here](#)). The goal of the characterization test is to verify the code's existing behavior, including side-effects, if any.
 - DeepSource provides *test coverage tracking*, with support for Javascript, Go, Python,

Ruby, and Java. The test coverage analyzer, once enabled, provides a report showing how much of the code is covered by tests and as well as issues caught by the tool.



- **Refactor the code.**

- Once tests are in place, it becomes easier to work with the code confidently, whether to port it to another language, extend it, or migrate it. Tests share immediate feedback as to whether or not the porting or refactoring is going in the right direction. Working with legacy code is also an opportunity to remove any accumulated technical debt that could pollute the update.
- DeepSource Analyzers, once configured, can act on both the legacy code to gain insights into potential areas of concern when the legacy code is updated and can be set up to continuously ensure that the updated code adheres to whatever specifications have been set by the team.

- **Space out incremental changes into several review cycles.**

- Set up the project such that it is divided into several phases, epics, or milestones (depending on your project management method). Have a review cycle – with integration or end-to-end tests – to catch issues that unit tests can't catch, such as issues with real-world user input, which aren't expected or easily accounted for.
- It is not recommended to push a codebase to live/production in its entirety but to replace the legacy code in parts if possible. The difficulty or ease of this endeavor depends on the system's architecture (compare, for example, microservices vs. monoliths.) Therefore, the way updates are rolled out should take the software architecture into account.

How does DeepSource Help?

DeepSource is an all-in-one solution that leverages existing, trusted libraries with its own in-house static code analyzers.

- Its *Transformers* support code formatters such as Black, YAPF, and gofmt
- Its *Analyzers* are a suite of tools that can gather data on test coverage, detect issues, auto-fix certain types of issues. DeepSource's static code analysis has a false positivity rate of less than 5%, which means it's a dependable tool for finding and solving bugs.
- DeepSource Analyzers support the most common languages used today: Python, Javascript, Java, Ruby, PHP, and Go.
- DeepSource is available for the most common repository providers: GitHub, GitLab, and BitBucket.
- It integrates with different CI tools such as TravisCI and CircleCI, as well as GitHub Actions.

Closing thoughts

There is no escaping legacy code. It has always been there and will always be there. Working with it can be an exercise in frustration – or a chance to learn from a previous project's successes and shortcomings. Knowing what it is and how to work with it properly will make all long-term projects moving forward easier for you and other developers.

Useful Links & References

- [What makes code "legacy?"](#) and its duplicate: [What is the definition of legacy code?](#)
- [Michael Feathers' Blog](#)
- [Unemployment systems – COBOL legacy software](#)